

```

1 #!/usr/bin/python
2 import os
3 import json
4 import sys
5 sys.path.insert(0, '../bluetooth')
6 import bluetooth_gap
7 import bluetooth_constants
8
9 if 'REQUEST_METHOD' in os.environ:
10     result = {}
11     args = json.load(sys.stdin)
12     if os.environ['REQUEST_METHOD'] != 'PUT':
13         print 'Status: 405 Method Not Allowed'
14         print
15         print "Status-Line: HTTP/1.0 405 Method Not Allowed"
16         print
17     else:
18         print "Content-Type: application/json;charset=utf-8"
19         print
20         if not "bdaddr" in args:
21             result['result'] = bluetooth_constants.RESULT_ERR_BAD_ARGS
22             print json.JSONEncoder().encode(result)
23         else:
24             bdaddr = args["bdaddr"]
25             rc = bluetooth_gap.connect(bdaddr)
26             result['result'] = rc
27             print json.JSONEncoder().encode(result)
28     else:
29         print "ERROR: Not called by HTTP"

```

do_connect.py

Fig. 9: Adapter code

```

10 #!/usr/bin/python
11 import bluetooth_constants
12 import bluetooth_general
13 import bluetooth_exceptions
14 import bluetooth_utils
15 import sys
16 import dbus
17 import dbus.mainloop.glib
18 from dbus import ByteArray
19
20
21 def connect(bdaddr):
22     bus = dbus.SystemBus()
23     device_proxy = bluetooth_general.getDeviceProxy(bus, bdaddr)
24     if device_proxy == None:
25         return bluetooth_constants.RESULT_ERR_NOT_FOUND
26     device_path = device_proxy.object_path
27     if not bluetooth_general.is_connected(bus, device_path):
28         try:
29             device_proxy.Connect()
30         except Exception as e:
31             return bluetooth_constants.RESULT_EXCEPTION
32         return bluetooth_constants.RESULT_OK
33     else:
34         return bluetooth_constants.RESULT_OK
35
36

```

Fig. 10: API code

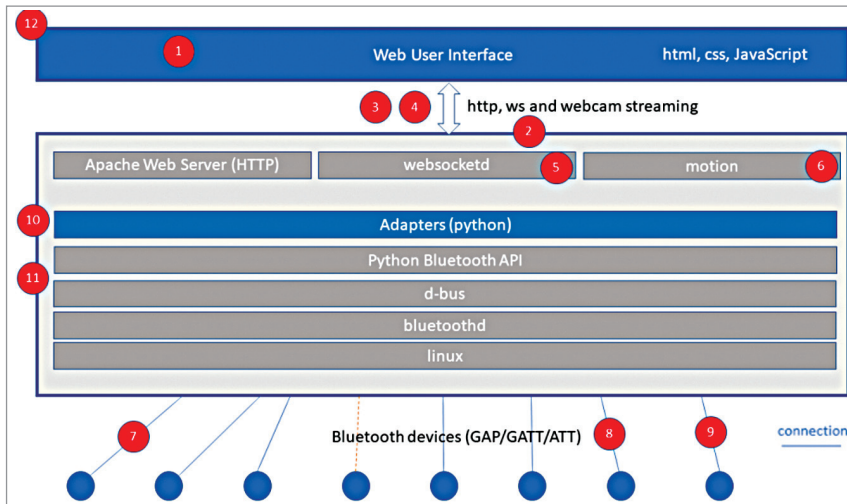


Fig. 11: Security analysis

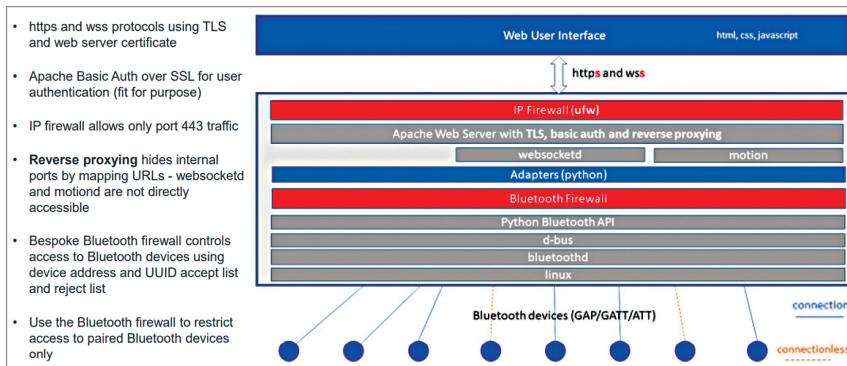


Fig. 12: Steps for securing the gateway

The gateway at this stage used three different TCP/IP ports: port 80 for HTTP, 8081 for the webcam streaming service, and 8082 for web-

socketd. The more ports exposed directly to the Internet, the greater is the security risk.

Data over TCP/IP is not encrypted

at this stage either and nor is data over Bluetooth necessarily.

It was also considered desirable that there be some way to grant access to specific Bluetooth devices only rather than all devices that happened to be in range of the gateway. Further, it would be advantageous to be able to grant access to only specific device capabilities rather than all things that a device can do over Bluetooth.

Reverse proxying

There were three services running on the gateway, each listening to a different port. But the installed IP firewall was configured to only allow port 443 access. The Apache web server was set up to act as a reverse proxy and forward port 443 packets to another port directed by the URL patterns (Fig. 13). This limited the number of ports exposed to the Internet.

In addition, the concept of a Bluetooth firewall was invented for the project. This allowed control over which Bluetooth devices could be interacted with via the gateway and which of their services, characteristics, and descriptors could be accessed. The configuration file for the Bluetooth firewall looks like as shown in Fig. 14.

Bluetooth Internet gateway scalability

Whilst the Bluetooth specifications do not stipulate a limit to the number of